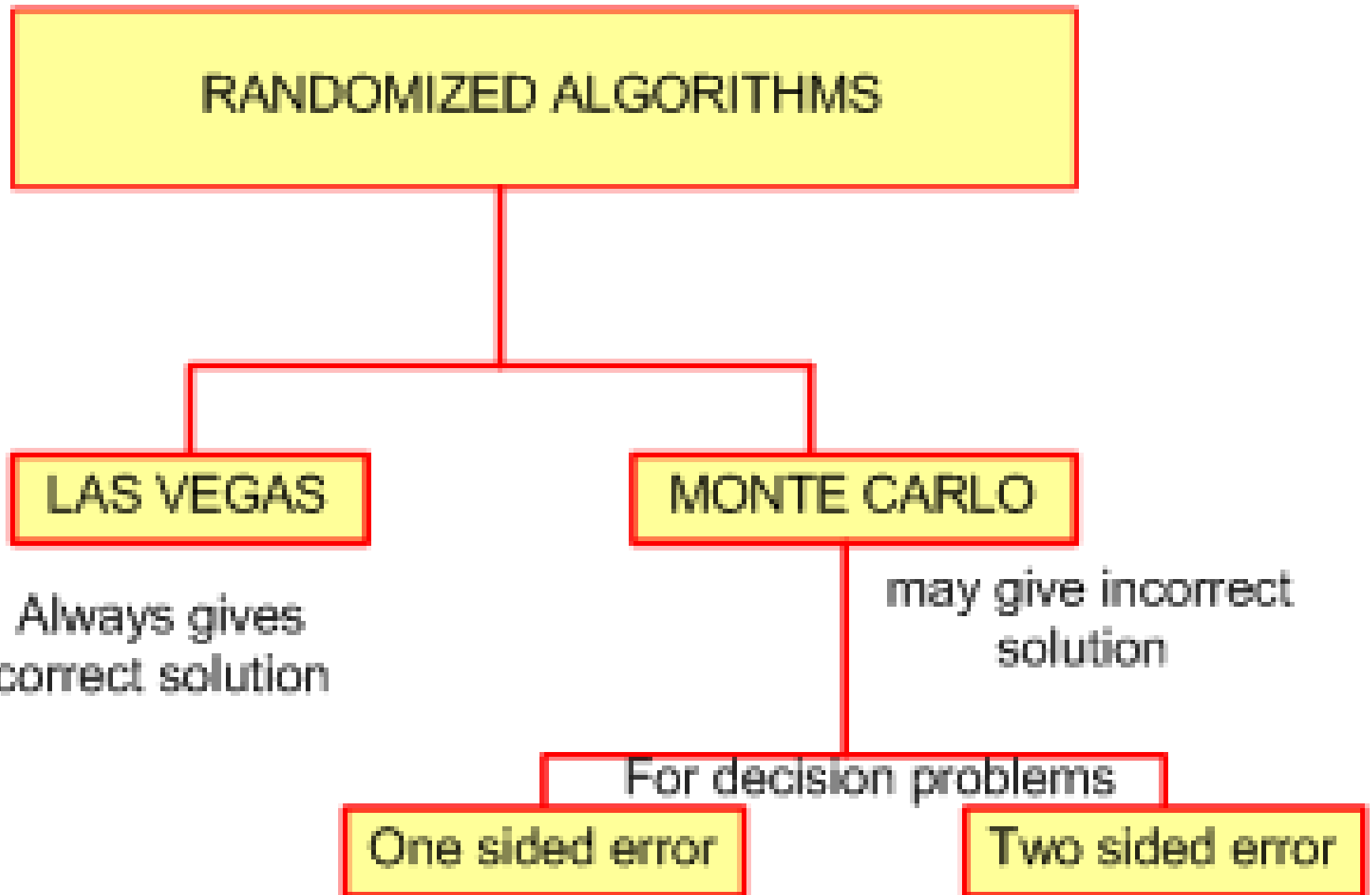


Randomized Algorithms



Monte Carlo & Las Vegas

```
def polyequal(A,B,d): "One-sided Monte Carlo Algorithm"
    equal = True
    for i in 1...k:
        x = randint(1, 100*d)
        equal = equal and A(x) == B(x)
    return equal
```

If $A \equiv B$, then
 $\Pr[\text{polyequal}(A,B,d)] = 1.$

If $A \not\equiv B$, then
 $\Pr[\text{polyequal}(A,B,d)] \leq 100^{-k}.$

```
def polyequal(A,B,d): "Las Vegas Algorithm" (w/o replacement)
    order = sample(range(1, 100*d), d+1)
    for x in order:
        if A(x) != B(x):
            return False
    return True
```

If $A \equiv B$, then
 $\Pr[\text{polyequal}(A,B,d)] = 1.$

If $A \not\equiv B$, then
 $\Pr[\text{polyequal}(A,B,d)] = 0.$

Randomized Algorithms

- Las Vegas:
 - Ex: Randomized Quick Sort
- Monte Carlo:
 - Ex: Min-Cut algorithm
 - One sided Error: If the probability that it errors is ZERO for at least one of the possible output (Yes or No)
 - Two sided Error: If there is a NON-ZERO probability that it errors when it outputs either YES or NO
- Las Vegas = Monte Carlo (*with the error probability Zero*)

Min Cut Algorithm

Consider a graph-theoretic example. Let G be a connected, undirected multigraph with n vertices. A *multigraph* may contain multiple edges between any pair of vertices. A *cut* in G is a set of edges whose removal results in G being broken into two or more components. A *min-cut* is a cut of minimum cardinality. We now study a simple algorithm for finding a min-cut of a graph.

We repeat the following step: pick an edge uniformly at random and merge the two vertices at its end-points (Figure 1.1). If as a result there are several edges between some pairs of (newly formed) vertices, retain them all. Edges between vertices that are merged are removed, so that there are never any self-loops. We refer to this process of merging the two end-points of an edge into a single vertex as the *contraction* of that edge. With each contraction, the number of vertices of G decreases by one. The crucial observation is that an edge contraction does not reduce the min-cut size in G . This is because every cut in the graph at any intermediate stage is a cut in the original graph. The algorithm continues the contraction process until only two vertices remain; at this point, the set of edges between these two vertices is a cut in G and is output as a candidate min-cut.

- Suppose u and v are vertices that are connected by an edge in some multigraph G . To collapse the edge $\{u, v\}$, we create a new node called uv , replace any edge of the form $\{u, w\}$ or $\{v, w\}$ with a new edge $\{uv, w\}$, and then delete the original vertices u and v .

```

MINCUT( $G$ )
Let  $v_1$  be any vertex of  $G$ 
 $n = |V(G)|$ 
 $S = \{v_1\}$ 
for  $i = 2$  to  $n$ 
    let  $v_i$  the node of  $V \setminus S$  s.t.
     $c(S : \{v\})$  is maximized (over all  $v \in V \setminus S$ )
     $S := S \cup \{v_i\}$ 
endfor
if  $n = 2$  then return the cut  $\delta(\{v_n\})$ 
else
    Let  $G'$  be obtained from  $G$  by shrinking  $v_{n-1}$  and  $v_n$ 
    Let  $C$  be the cut returned by MINCUT( $G'$ )
    Among  $C$  and  $\delta(\{v_n\})$  return the smaller cut (in terms of
cost)
endif

```

probability of picking an edge

- If we pick an edge in G uniformly at random, the probability of picking an edge in the minimum cut is at most $2/n$
- The overall probability $P(n)$ that returns the true minimum cut is given by the recurrence

$$P(n) \geq \frac{n-2}{n} \cdot P(n-1)$$

$$P(2) = 1$$

$$P(n) \geq \prod_{i=3}^n \frac{i-2}{i} = \frac{\prod_{i=3}^n (i-2)}{\prod_{i=3}^n i} = \frac{\prod_{j=1}^{n-2} j}{\prod_{i=3}^n i} = \boxed{\frac{2}{n(n-1)}}$$

Karger Min Cut

- Run the guessing algorithm many times and return the smallest guess.
- Only way for the Karger algorithm to return the wrong answer is if Karger-Min-Cut Algo fails N times in a row.

KARGERMINCUT(G):

$min_k \leftarrow \infty$

for $i \leftarrow 1$ to N

$X \leftarrow \text{GUESSMINCUT}(G)$

if $|X| < min_k$

$min_k \leftarrow |X|$

$minX \leftarrow X$

return $minX$

Probability of success

- Each guess is independent, so probability of success is given as

$$1 - \left(1 - \frac{2}{n(n-1)}\right)^N \leq 1 - e^{-2N/n(n-1)},$$

When N is larger, this probability arbitrarily reaches close to 1, but never equal to 1

If we set N value $N = c \binom{n}{2} \ln n$

The World's most useful inequality :

$$1 + x \leq e^x$$

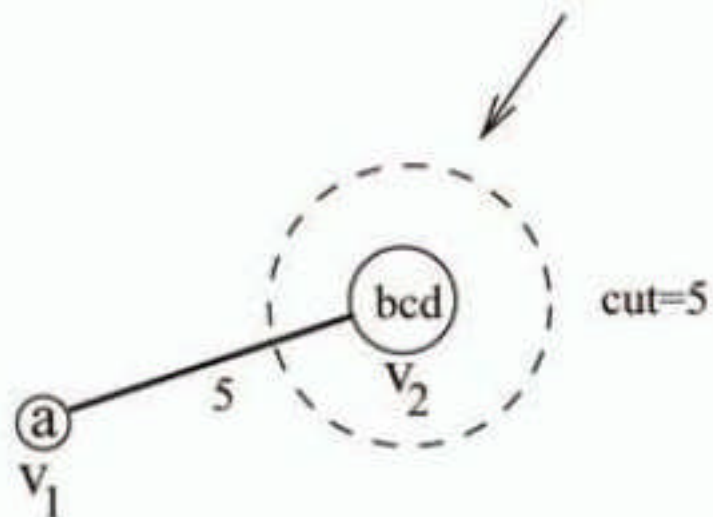
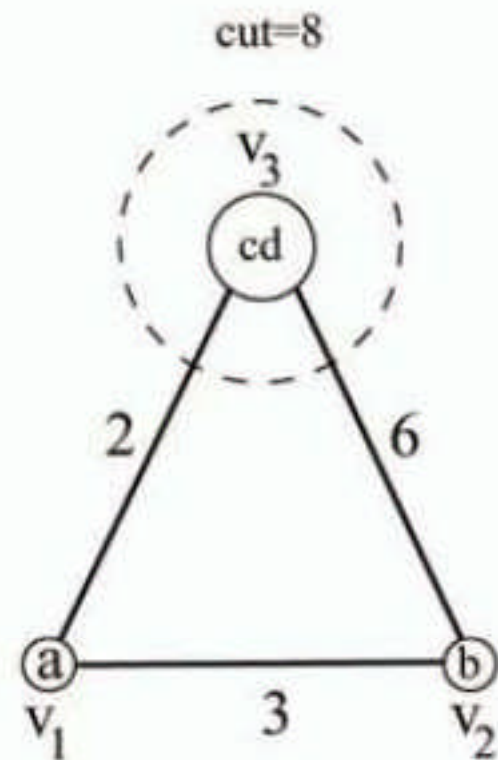
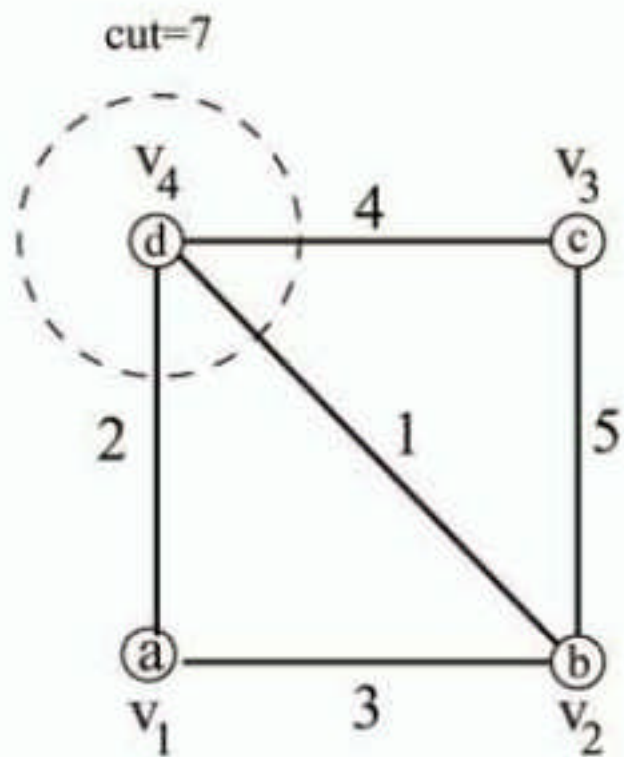
The correct probability can be given as:

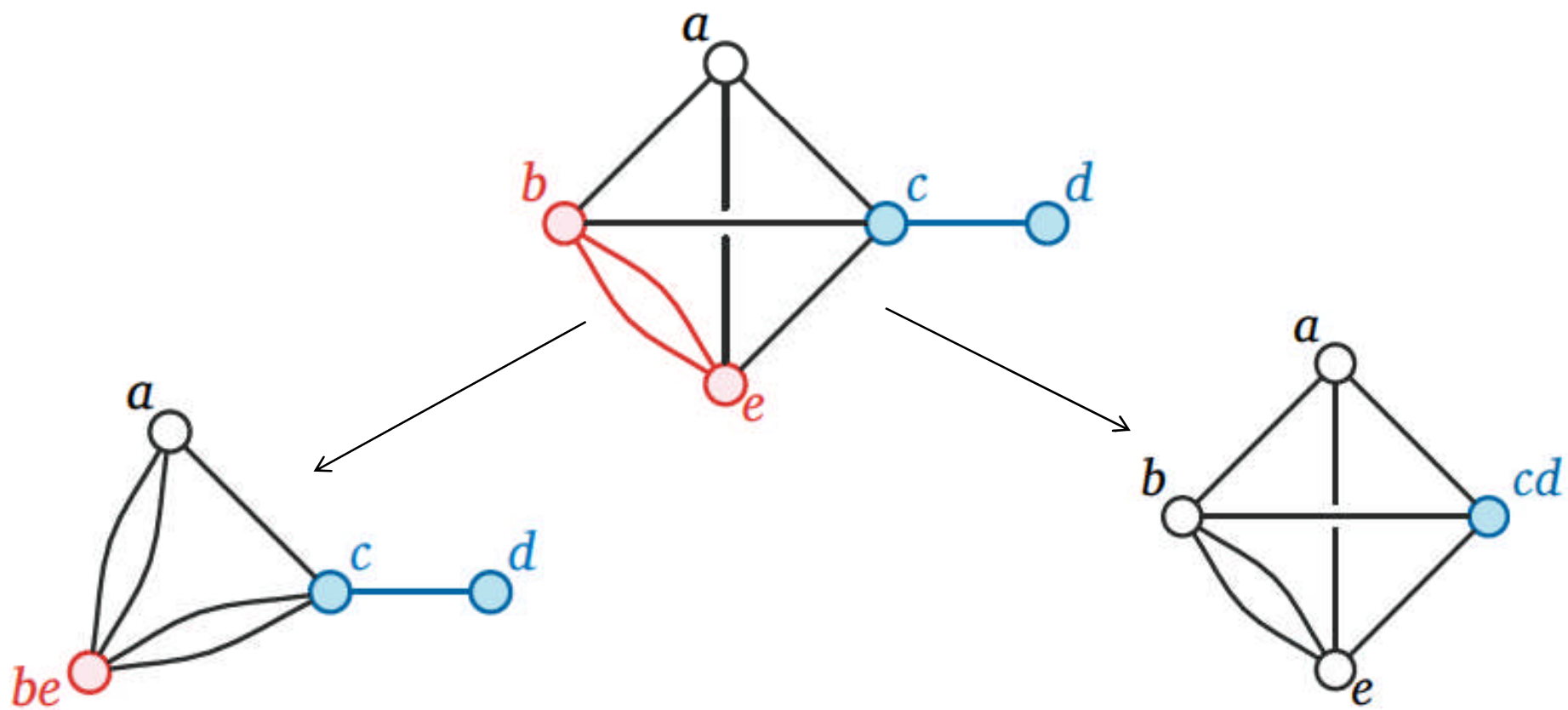
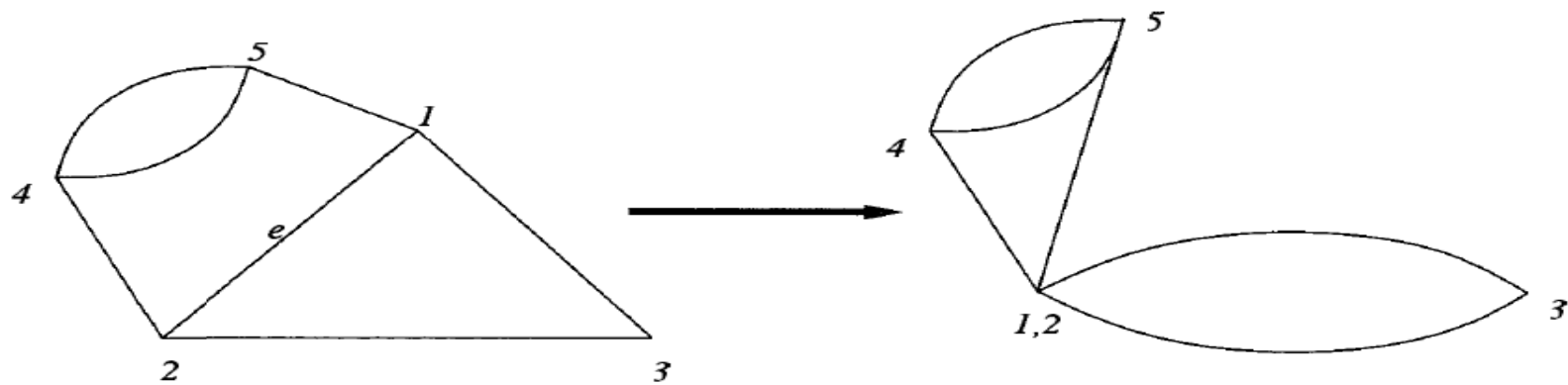
$$1 - e^{-c \ln n} = 1 - \frac{1}{n^c}$$

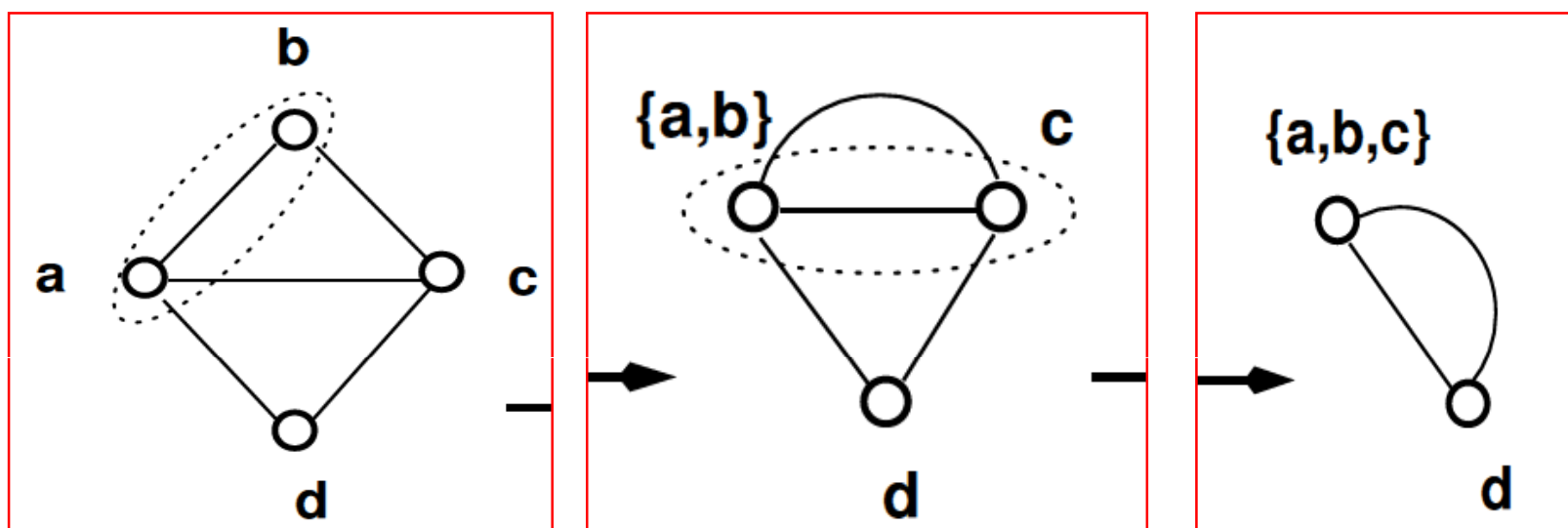
The complexity is : $O(n^4 \log n)$ time

Better probability

- When the failure probability is a polynomial fraction, we say that the algorithm is correct with **high probability**.
- If we make the number of iterations even larger $N = n^2(n-1)/2$
- Then the success probability is $1 - e^{-n}$







A Minimum Cut Algorithm

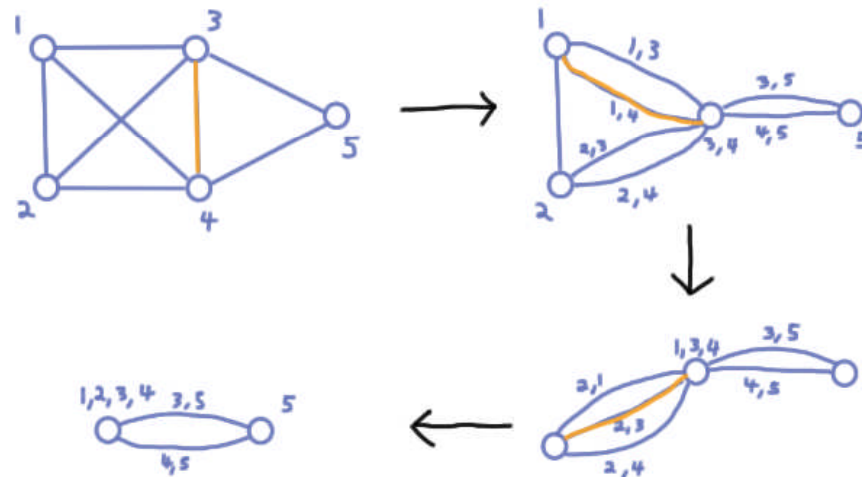
Given: A connected graph $G=(V,E)$ (S is a minimum "cut-set")

Output: A set S of minimum size such that $(V,E-S)$ has two connected components.

```
def randMinCutSet(V,E):  
  While  $|V| > 2$ :
```

```
     $(V,E) = \text{ContractEdge}(V,E,e)$ 
```

```
  return E
```



Global min-cut

- if any node v had degree less than k , then the cut $(\{v\}; V - \{v\})$ would have size less than k , contradicting our assumption that $(A; B)$ is a global min-cut.
- Every node in G has degree at least k

$$|E| \geq \frac{1}{2}kn$$

- the probability that an edge in F is contracted is at most

$$\frac{k}{\frac{1}{2}kn} = \frac{2}{n}.$$

Example: Monte Carlo algorithm

- Consider a Monte Carlo algorithm A for a problem Π whose expected running time is at most $T(n)$ on any instance of size n and that produces a correct solution with probability $y(n)$. Suppose further that given a solution to Π , we can verify its correctness in time $t(n)$.
Show how to obtain a Las Vegas algorithm that always gives a correct answer to Π and runs in expected time at most $(T(n) + t(n))/y(n)$.

